

MINI PROJECT

PROJECT TITLE :

Task~1 :

Creation of S3 Bucket, Enabling versioning, hosting static website, updating website, and rolling back to previous version.

Task~2 :

Creation of two EC2 web servers – one with Apache and one with Nginx, with installation done using user data.

SUBMITTED BY : Lokeshwar Reddy kukkala

DOMAIN : AWS Cloud computing

SUBMITTED TO : SkillForge Company

DATE OF SUBMISSION : 01/12/2025

TASK 1 : S3 STATIC WEBSITE HOSTING WITH VERSIONING AND ROLLBACK.

INTRODUCTION :



Cloud computing provides scalable, reliable, and cost-effective solutions for storing and delivering data over the internet. One of the most widely used cloud storage services offered by Amazon Web Services (AWS) is Amazon Simple Storage Service (S3). Amazon S3 is highly durable, secure, and is commonly used for data storage, backup, and static website hosting.

In this task, a static website is hosted using Amazon S3 without using any physical server or virtual machine. The task also demonstrates the use of S3 Versioning, which helps in maintaining multiple versions of files and enables easy rollback and recovery in case of accidental deletion or modification.

This experiment gives practical knowledge of how modern websites can be hosted securely and efficiently using cloud storage services

AIM OF THE EXPERIMENT :

The main aim of this task is to create an Amazon S3 bucket, host a static website viewing, implement versioning for backup and recovery, and demonstrate how rollback works using multiple versions of the same file.

TOOLS AND SERVICES USED :

- Amazon Web Services (AWS)
- Amazon S3 (Simple Storage Service)
- Web Browser
- HTML Files

FILES UPLOADED IN S3 BUCKET :

The following files were uploaded into the S3 bucket for static website hosting :

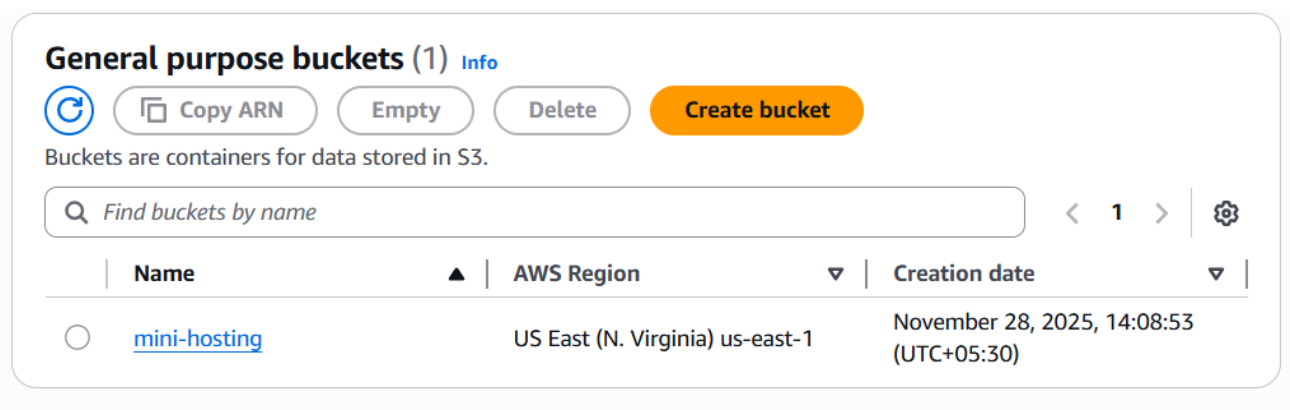
1. Index.html = Main homepage of the website.
2. Error.html = Error page displayed for incorrect URLs.
3. Image.jpg = Image file used in the website.

All required files for static website hosting were placed inside the same S3 bucket.

STEP-BY-STEP PROCEDURE :

STEP 1 : CREATION OF S3 BUCKET

An S3 bucket with the name “ **mini-hosting** ” was created in AWS. This bucket is used to store all the website files.



➤ **S3 Bucket Name** : mini-hosting

STEP 2 : UPLOADING WEBSITE FILES

The uploaded Files in the S3 bucket [mini-hosting] are :

- Index.html
- Error.html
- Image.jpg

Objects (3)		Copy S3 URI	Copy URL	Download	Open	Delete	Actions	Create folder	Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more									
Find objects by prefix		☐ Show versions		< 1 > Settings					
☐	Name	Type	Last modified	Size	Storage class				
☐	error.html	html	November 28, 2025, 14:19:20 (UTC+05:30)	126.0 B	Standard				
☐	image.jpeg	jpeg	November 28, 2025, 14:19:23 (UTC+05:30)	114.7 KB	Standard				
☐	index.html	html	November 28, 2025, 14:45:41 (UTC+05:30)	255.0 B	Standard				

Were uploaded into the S3 Bucket. These files are required for hosting the static website properly.

STEP 3 : ENABLING STATIC WEBSITE HOSTING

Static website hosting was enabled from the bucket properties. The following configuration was used :

- Index Document : index.html
- Error Document : error.html

S3 static website hosting

Enabled

Hosting type

Bucket hosting

Bucket website endpoint

When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)

<http://mini-hosting.s3-website-us-east-1.amazonaws.com>

After enabling this feature, AWS provided a “ **Bucket Website Endpoint URL** ”, which is used to access the hosted website.

STEP 4 : ALLOWING PUBLIC ACCESS

To make the website accessible to everyone on the internet :

- Block Public Access was turned **OFF**.

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, and IAM permissions. By default, public access is only to this bucket and its access points. AWS recommends that you turn on Block all public access for your buckets or objects within, you can customize the individual settings below to suit your specific needs.

Block *all* public access

 Off

► Individual Block Public Access settings for this bucket

- A Bucket Policy was added to allow public read-only access to website files.

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts.

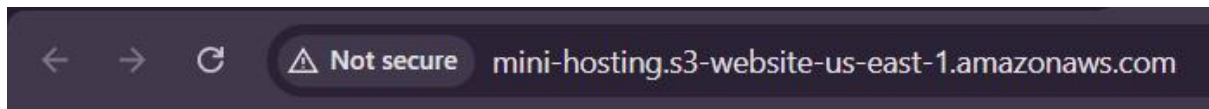
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mini-hosting/*"
    }
  ]
}
```

Using bucket policy, access can also be restricted to specific users if required, providing better security control.

STEP 5 : WEBSITE TESTING

The Static Website endpoint provided by AWS was opened in a browser :

- If the URL is correct , index.html is displayed.

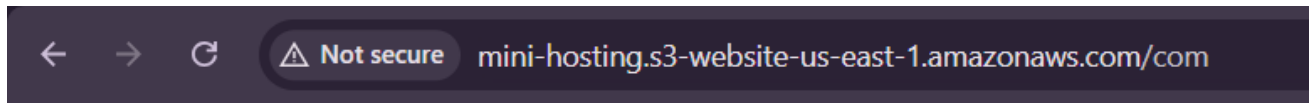


I Love Marvel Movies

Hello Tony Stark



- If the URL is incorrect, error.html is displayed.



Sorry, your static website url is not correct

This confirms that static website hosting is working successfully.

ENABLING VERSIONING :

Versioning was enabled on the same S3 bucket to maintain multiple versions of files. This feature helps in recovering previous versions of files if they are modified or deleted by mistake.

Bucket Versioning

Versioning is a means of keeping multiple variants of an object. It can easily recover from both unintended user actions and deletions.

Bucket Versioning
Enabled

mini-hosting [info](#)

Objects Metadata Properties Permissions Metrics Management Access Points

Objects (3) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open ↗](#) [Delete](#) [Actions ▼](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☒ Show versions < 1 > [Settings](#)

☐	Name	Type	Version ID	Last modified	Size	Storage class
☐	error.html	html	null	November 28, 2025, 14:19:20 (UTC+05:30)	126.0 B	Standard
☐	image.jpeg	jpeg	null	November 28, 2025, 14:19:23 (UTC+05:30)	114.7 KB	Standard
☐	index.html	html	null	November 28, 2025, 14:45:41 (UTC+05:30)	255.0 B	Standard

After enabling the Bucket version it will show version id for each file. But for current files it will show version id is **NULL**.

UPDATING INDEX FILE :

After enabling versioning, the same “ **index.html** ”file was modified with a small change and uploaded again into the same S3 bucket with the same name. AWS automatically created a new Version ID for the updated file.

Objects (4) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open ↗](#) [Delete](#) [Actions ▼](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

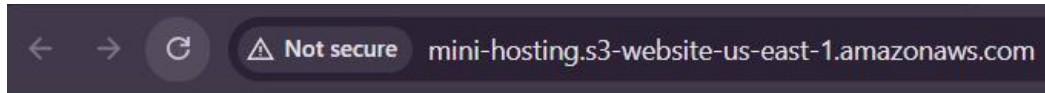
☒ Show versions < 1 > [Settings](#)

☐	Name	Type	Version ID	Last modified	Size	Storage class
☐	error.html	html	null	November 28, 2025, 14:19:20 (UTC+05:30)	126.0 B	Standard
☐	image.jpeg	jpeg	null	November 28, 2025, 14:19:23 (UTC+05:30)	114.7 KB	Standard
☐	index.html	html	svm3kAIKD_g5bqzmz31 OUDQfjtavOCU.5	November 28, 2025, 14:59:01 (UTC+05:30)	252.0 B	Standard
☐	index.html	html	null	November 28, 2025, 14:45:41 (UTC+05:30)	255.0 B	Standard

Now it will appear the New Version ID for new updated index.html. After enabling the Version ID will appear.

When the static website endpoint URL was refreshed :

- The old “ index.html ” was replaced by the new updated version.



I Love Tony Stark

Hello Tony Stark



After refreshing the old index.html , this is the New updated version of index.html in the S3 bucket.

ROLLBACK USING VERSIONING :

When the newer version of the modified index.html file was deleted :

- Only the latest version was removed.
- The previous version of index.html was automatically restored.

mini-hosting [Info](#)

[Objects](#) [Metadata](#) [Properties](#) [Permissions](#) [Metrics](#) [Management](#) [Access Points](#)

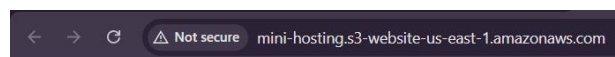
Objects (3) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open i2](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

[Show versions](#) < 1 >

☐	Name	Type	Version ID	Last modified	Size	Storage class
☐	error.html	html	null	November 28, 2025, 14:19:20 (UTC+05:30)	126.0 B	Standard
☐	image.jpeg	jpeg	null	November 28, 2025, 14:19:23 (UTC+05:30)	114.7 KB	Standard
☐	index.html	html	null	November 28, 2025, 14:45:41 (UTC+05:30)	255.0 B	Standard

After refreshing the website URL again :



I Love Marvel Movies

Hello Tony Stark



- The website displayed the older original web page.

This confirms that rollback functionally using S3 versioning works correctly.

RESULT :

The static website was successfully hosted using AWS S3. Versioning was enabled and tested effectively. The rollback functionality was also verified by restoring the previous version of the website after deleting the latest version.

CONCLUSION :

The Task 1 demonstrate how AWS S3 bucket can be used for static website hosting along with versio control. The experiment also proves that versioning is a powerful feature for backup, recovery, and data protection in cloud storage. This setup ensures high availability, safety, and reliability of website data.

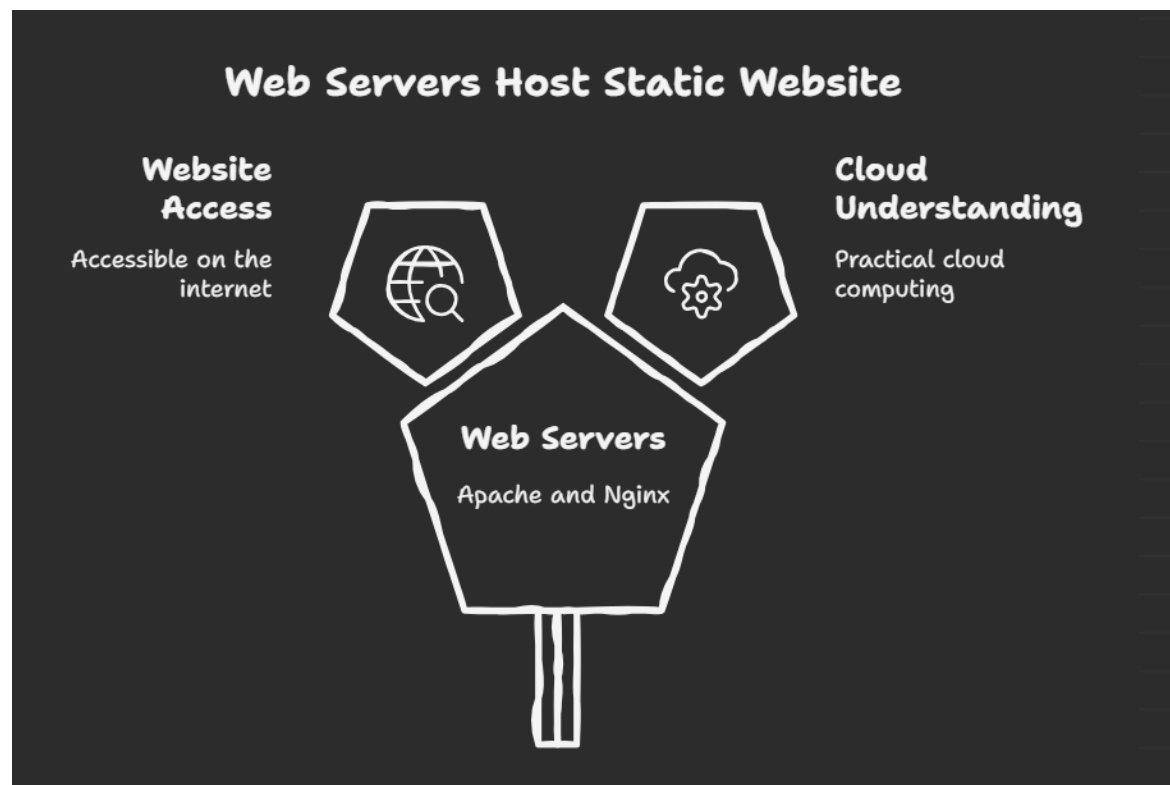
TASK 2 : HOSTING A STATIC WEBSITE USING APACHE & NGINX ON AWS EC2

Introduction :

Web hosting is the process of making a website accessible on the internet using a web server. In this project, I hosted a static website using two popular web servers :

“APACHE & NGINX” on an AWS EC2 instances.

Apache and Nginx are widely used open-source web servers that serve web content to users through HTTP protocol. This project helped me gain practical knowledge of cloud computing and web server deployment.



OBJECTIVE OF THE PROJECT :

The main objective of this project are :

- To launch an EC2 instance on AWS.
- To install and configure Apache Web Server.
- To install and configure Nginx Web Server.
- To host a static website using both servers.
- To access the website using a public IPV4 address

REQUIRMENTS :

1. AWS Account
2. EC2 Instance (Amazon Linux)
3. Apache Web Server (httpd)
4. Nginx Web Server
5. Static HTML file (index.html)
6. Internet Browser
7. AWS Terminal

PART – A : HOSTING WEBSITE USING APACHE WEB SERVER.

Apache Server implementation

STEP 1 : Launch EC2 Instance

- Created an EC2 instance using Amazon Linux.
- Instance type is t3.micro

Instances (1) [Info](#)

Find Instance by attribute or tag (case-sensitive)

Instance state = running [Clear filters](#)

Connect Instance state Actions Launch instances

All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
apache-server	i-00801ceb1e86ebcd1	Running	t3.micro	3/3 checks passed	View alarms	us-east-1f	ec2-13-221-

i-00801ceb1e86ebcd1 (apache-server)

[Instance summary](#) [Info](#)

Instance ID i-00801ceb1e86ebcd1	Public IPv4 address 13.221.57.190 open address	Private IPv4 addresses 172.31.65.201
IPv6 address -	Instance state Running	Public DNS ec2-13-221-57-190.compute-1.amazonaws.com open address

- Launched and connected to the EC2 instance.
- EC2 Instance Name : apache-server

STEP 2 : USER DATA

```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y httpd
```

```
systemctl start httpd
```

```
systemctl enable httpd
```

Now, the user data is install the apache web server in EC2 instance [apache-server].

STEP 3 : SECURITY GROUPS

The Allowed ports in the security group are :

- SSH (22) for remote access.

- HTTP (80) for web traffic.

Details			
Security group name  my-security-group	Security group ID  sg-0dab3d93bf3a164c5	Description  Allow ssh access to developers	VPC ID  vpc-02b49aee05dd92ced 
Owner  539609142189	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

sg-0dab3d93bf3a164c5 - my-security-group

Inbound rules (2)

Q Search

Manage tags

Edit inbound rules

< 1 >

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0273021aef6806c2	IPv4	HTTP	TCP	80
☐	-	sgr-068b8aa0588be9662	IPv4	SSH	TCP	22

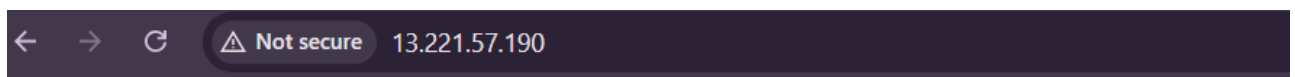
➤ Security Group Name : my-security-group

STEP 4 : Access Apache Website

Opened browser and entered :

<http://13.221.57.190>

Now in browser, it will appear small static website.



Welcome to Vikki's Apache Web Server

This static page is hosted on an EC2 instance using Apache (installed via User Data).

Instance public IP:

Task: Demonstrate Apache hosting a static site.

The website successfully displayed using Apache server.

OUTPUT :

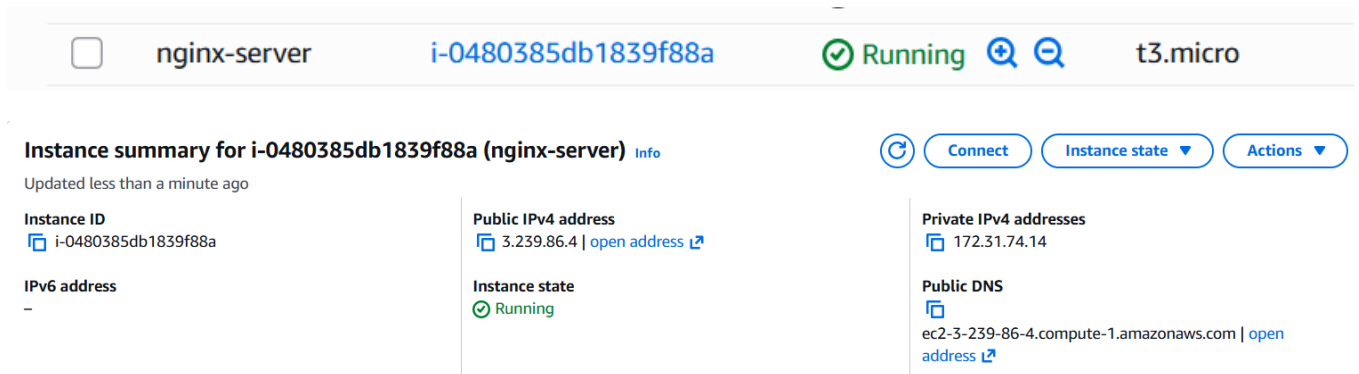
The Static Website was hosted successfully using Apache on AWS EC2, and it displayed the custom HTML webpage through the public IPv4 address.

PART – B : HOSTING WEBSITE USING NGINX WEB SERVER.

Nginx Server implementation

STEP 1 : Launch EC2 Instance

- Created an EC2 instance using Amazon Linux.
- Instance type is t3.micro.



The screenshot displays the AWS Management Console interface for an EC2 instance. At the top, a summary bar shows the instance name 'nginx-server', its ID 'i-0480385db1839f88a', its state 'Running' with a green checkmark, and its instance type 't3.micro'. Below this, the 'Instance summary for i-0480385db1839f88a (nginx-server)' is shown, updated less than a minute ago. The summary is divided into three columns: 'Instance ID' with the value 'i-0480385db1839f88a', 'Public IPv4 address' with the value '3.239.86.4' and a link to 'open address', and 'Private IPv4 addresses' with the value '172.31.74.14'. The 'Instance state' is 'Running' with a green checkmark. The 'Public DNS' is 'ec2-3-239-86-4.compute-1.amazonaws.com' with a link to 'open address'. On the right side of the summary, there are buttons for 'Connect', 'Instance state', and 'Actions'.

Instance ID	Public IPv4 address	Private IPv4 addresses
i-0480385db1839f88a	3.239.86.4 open address	172.31.74.14

Instance state: Running

Public DNS: [ec2-3-239-86-4.compute-1.amazonaws.com | open address](#)

- Launched and connected to the EC2 instance.
- EC2 Instance Name : nginx-server.

STEP 2 : USER DATA

```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y nginx
```

```
systemctl start nginx
```

```
systemctl enable nginx
```

Now, the user data is install the nginx web server in EC2 instance [nginx-server].

STEP 3 : SECURITY GROUPS

The Allowed ports in the security group are :

- SSH (22) for remote access.
- HTTP (80) for web traffic.

Details

Security group name  my-security-group	Security group ID  sg-0dab3d93bf3a164c5	Description  Allow ssh access to developers	VPC ID  vpc-02b49aee05dd92ced 
Owner  539609142189	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

sg-0dab3d93bf3a164c5 - my-security-group

Inbound rules (2)  [Manage tags](#) [Edit inbound rules](#)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0273021aef6806c2	IPv4	HTTP	TCP	80
☐	-	sgr-068b8aa0588be9662	IPv4	SSH	TCP	22

➤ Security Group Name : my-security-group

STEP 4 : Access Nginx Website

Opened browser and entered :

<http://3.239.86.4>

Now in browser, it will appear small static website.



Hello...! I am learning AWS Cloud Computing

This website is hosted using Nginx on an EC2 instance.

The website successfully displayed using Nginx server.

OUTPUT :

The Static Website was hosted successfully using Nginx on AWS EC2, and it displayed the custom HTML webpage through the public IPv4 address.

CONCLUSION :

In task 2 project, I successfully hosted a static website using two different web servers : Apache and Nginx on an AWS EC2 instance. This project helped me understand cloud deployment, Linux commands, webserver configuration, and real-world hosting techniques. It enhanced my practical skills in cloud computing and server management.